



Departamento de Informática e Matemática Aplicada – DIMAp

Padrões de Resiliência com Resilience4j Parte 2

Prof. Nélio Cacho

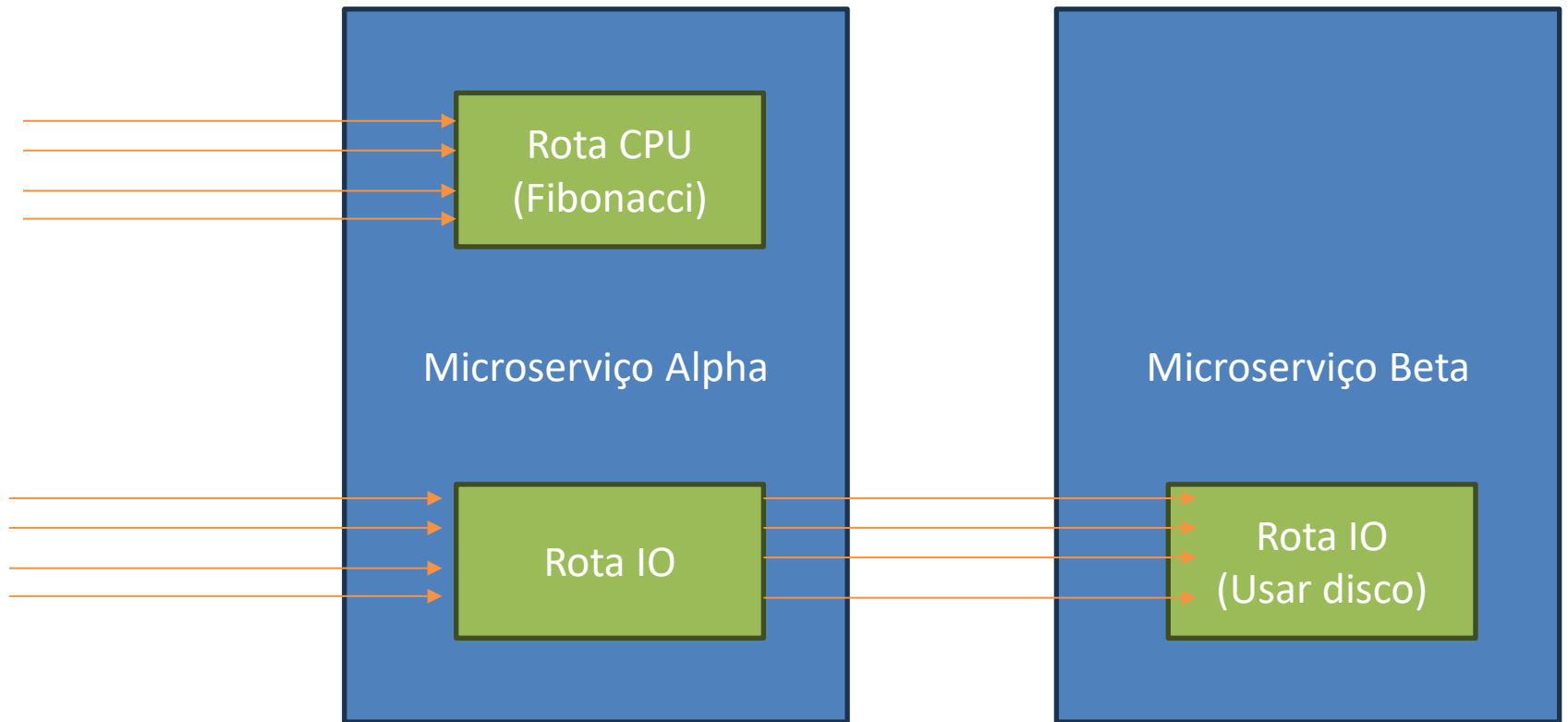
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

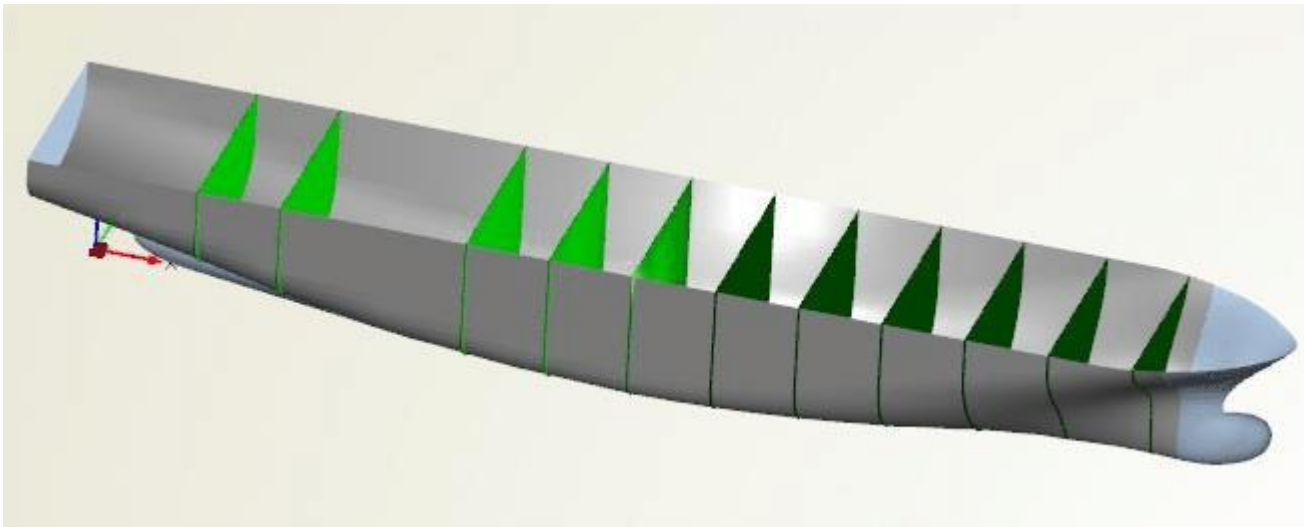
Problema!!!



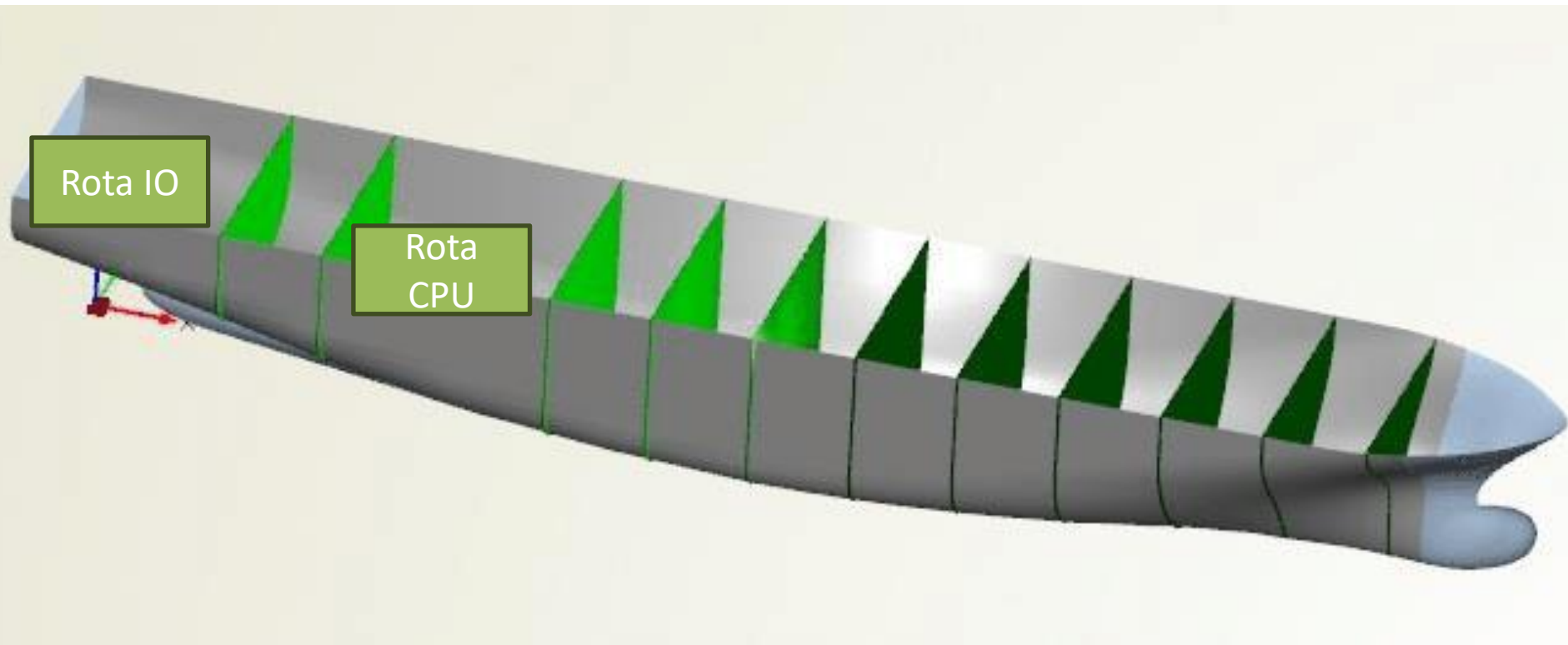
Problema

- Em uma aplicação baseada em microservices, frequentemente precisamos chamar vários microservices para concluir uma tarefa específica.
- Sem utilizar o padrão de bulkhead, o comportamento padrão dessas chamadas é executá-las usando as mesmas threads reservadas para lidar com solicitações para todo o contêiner Java.
- Em volumes altos, problemas de desempenho com um serviço entre muitos podem resultar em todas as threads do contêiner Java atingindo seu limite máximo e aguardando para processar o trabalho, enquanto novas solicitações de trabalho se acumulam.
- Neste casos, o contêiner Java eventualmente falhará.

Padrão Bulkhead



Solução

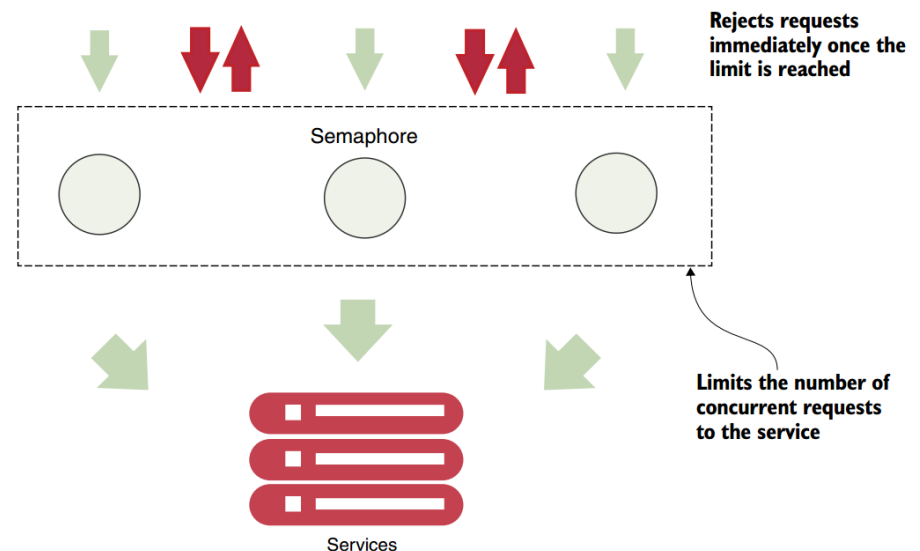


Padrão Bulkhead

- O padrão bulkhead separa as chamadas de recursos remotos em seus próprios pools de threads para que um único serviço com comportamento inadequado possa ser isolado e não cause o travamento do contêiner.
- O Resilience4j oferece duas implementações diferentes do padrão de bulkhead.

Padrão Bulkhead: Semaphore

- Bulkhead baseado em semáforo - Utiliza uma abordagem de isolamento baseada em semáforo, limitando o número de solicitações simultâneas ao serviço.
- Uma vez que o limite é atingido, ele começa a rejeitar as solicitações. Por padrão, o Resilience4j utiliza o tipo de bulkhead baseado em semáforo.



Padrão Bulkhead: Semaphore

- Esse modelo funciona bem se tivermos um pequeno número de recursos remotos sendo acessados dentro de uma aplicação e os volumes de chamadas para os serviços individuais estiverem distribuídos de forma relativamente uniforme.
- O problema surge quando temos serviços com volumes muito maiores ou tempos de conclusão mais longos do que outros serviços, o que pode levar à exaustão de threads em nossos pools de threads, pois um serviço acaba dominando todas as threads no pool de threads padrão.

Usando Bulkhead Semaphore

```
@CircuitBreaker(name = "servicebeta",  
    fallbackMethod= "buildFallbackGetData")  
@Bulkhead(name= "bulkheadservicebeta",  
    fallbackMethod= "buildFallbackGetData")  
  
public ResponseEntity getData() {  
    return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);  
}  
  
public ResponseEntity buildFallbackGetData(Throwable t) {  
    return ResponseEntity.ok("Fallback in action");  
}
```

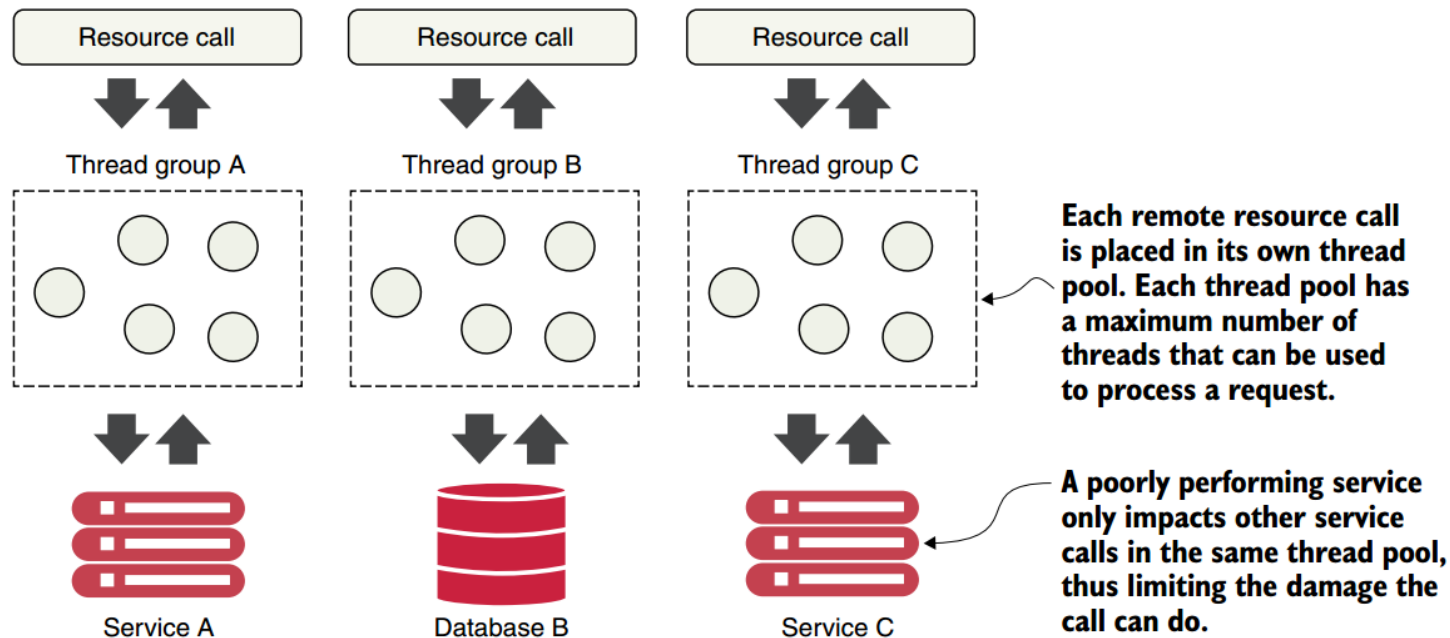
```
resilience4j.bulkhead:  
  instances:  
    bulkheadservicebeta:  
      maxWaitDuration: 2ms  
      maxConcurrentCalls: 20
```

Usando Bulkhead Semaphore

- **maxConcurrentCalls** - Define o número máximo de chamadas simultâneas permitidas pelo bulkhead. O valor padrão é 25. Se esse limite for atingido, novas requisições não entram imediatamente no processamento e irão para a espera.
- **maxWaitDuration** - Esse é o tempo máximo que uma thread pode ficar esperando para entrar no bulkhead quando ele está cheio. O valor padrão é 0 (Falha imediata).
 - Se o bulkhead estiver cheio, a thread espera até x segundos por uma vaga. Se nenhuma vaga abrir nesse tempo, a requisição falha

Padrão Bulkhead: Thread pool

- Bulkhead baseado em pool de threads - Utiliza uma fila limitada e um pool de threads fixo. Nessa abordagem, uma solicitação é rejeitada apenas quando o pool e a fila estão cheios.



Usando Bulkhead Thread pool

```
@CircuitBreaker(name = "servicebeta",
    fallbackMethod= "buildFallbackGetData")
@Bulkhead(name= "bulkheadservicebeta", type = Bulkhead.Type.THREADPOOL,
    fallbackMethod= "buildFallbackGetData")

public ResponseEntity getData() {
    return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);
}

public ResponseEntity buildFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action");
}
```

```
resilience4j.thread-pool-bulkhead:
  instances:
    bulkheadservicebeta:
      maxThreadPoolSize: 1
      coreThreadPoolSize: 1
      queueCapacity: 1
```

Usando Bulkhead Thread pool

- **maxThreadPoolSize** - Define o tamanho máximo do pool de threads. O valor padrão é `Runtime.getRuntime().availableProcessors()`.
- **coreThreadPoolSize** - Número de threads que ficam sempre ativas no pool. O valor padrão é `Runtime.getRuntime().availableProcessors()`.
- **queueCapacity** - Define a capacidade da fila. O valor padrão é 100.
- **keepAliveDuration** - Define o tempo máximo que as threads ociosas esperarão por novas tarefas antes de encerrar. Isso acontece quando o número de threads é maior do que o número de threads ativas. O valor padrão é 20 ms.

Usando Bulkhead Thread pool

- Qual é o dimensionamento adequado para um pool de threads personalizado? Para responder a essa pergunta, você pode usar a seguinte fórmula:
 - Tamanho do pool de threads = (número de solicitações por segundo no pico quando o serviço está saudável * latência em segundos) + uma pequena quantidade de threads extras para sobrecarga.

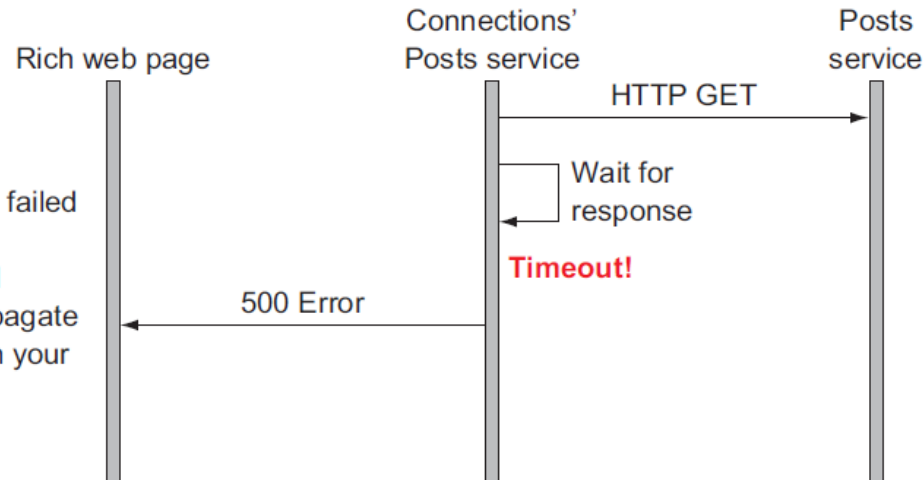
Padrão Retry

- O padrão de Retry é responsável por fazer novas tentativas de comunicação com um serviço quando esse serviço falha inicialmente.
- O conceito chave por trás desse padrão é fornecer uma maneira de obter a resposta esperada ao tentar invocar o mesmo serviço uma ou mais vezes, apesar da falha (por exemplo, uma interrupção de rede).
- Para esse padrão, devemos especificar o número de tentativas de retry para uma determinada instância de serviço e o intervalo desejado entre cada retry.

Padrão Retry

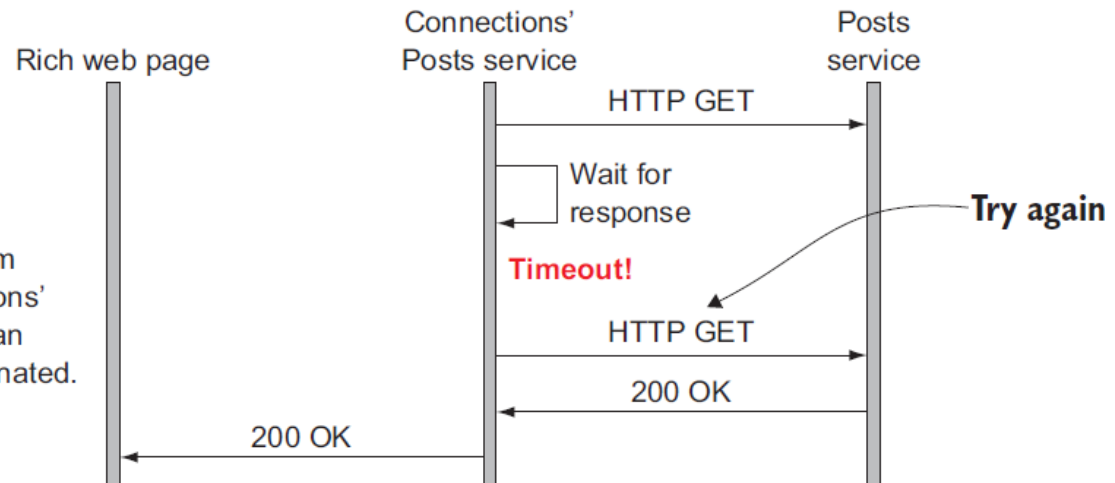
Without retries

Without retries, a single failed request between the Connections' Posts and Posts services can propagate the error further through your distributed system.

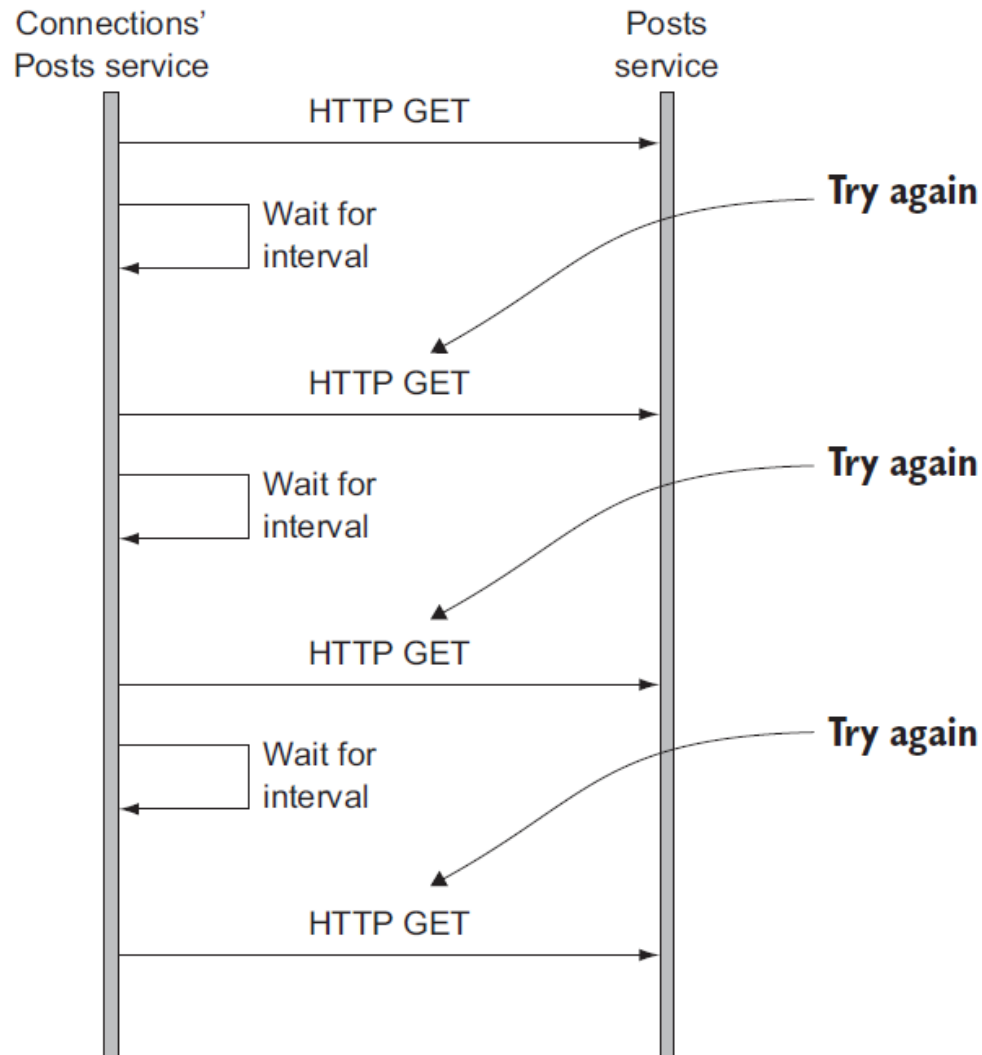


With retries

With retries, an intermittent problem between Connections' Posts and Posts can effectively be eliminated.



Cuidado com o Padrão Retry para evitar o *retry storm*



Padrão Retry

- `maxAttempts`: O número máximo de tentativas (incluindo a chamada inicial como a primeira tentativa).
- `waitDuration`: Uma duração fixa de espera entre as tentativas de nova execução.
- `intervalFunction`: Uma função para modificar o intervalo de espera após uma falha. Por padrão, a duração de espera permanece constante.

Padrão Retry

```
@CircuitBreaker(name = "servicebeta", fallbackMethod= "buildFallbackGetData")
@Retry(name = "retryservicebeta",
        fallbackMethod="retryFallbackGetData")
@Bulkhead(name= "bulkheadservicebeta", type = Bulkhead.Type.SEMAPHORE)
public ResponseEntity getData() {
    return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);
}

public ResponseEntity buildFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action for circuit breaker"+t.getMessage());
}

public ResponseEntity retryFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action for retry:"+t.getMessage());
}

public ResponseEntity bulkheadFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action for bulkhead:"+t.getMessage());
}
```

Padrão Retry

```
resilience4j.retry:  
  instances:  
    retryservicebeta:  
      maxAttempts: 5  
      waitDuration: 10000  
      intervalFunction:  
        type: exponential  
        exponential-backoff-multiplier: 2  
        max-interval: 2000ms  
      retry-exceptions:  
        - java.util.concurrent.TimeoutException  
        - java.net.ConnectException
```

Rate Limit

- O rate limit (limite de taxa) é uma técnica usada para controlar a quantidade de solicitações que um sistema pode receber ou processar durante um determinado período de tempo.
- O rate limiting pode ajudar a tornar sua API mais confiável nos seguintes cenários:
 - Um dos seus usuários está causando um aumento repentino no tráfego, e você precisa manter a disponibilidade para todos os outros.
 - Um dos seus usuários possui um script mal comportado que está enviando acidentalmente muitas solicitações.
 - Ou, pior ainda, um dos seus usuários está tentando intencionalmente sobrecarregar seus servidores.
 - Um usuário está enviando muitas solicitações de baixa prioridade, e você quer garantir que isso não afete o tráfego de alta prioridade. Por exemplo, usuários enviando um alto volume de solicitações para dados de análise podem afetar transações críticas de outros usuários.
 - Algo deu errado internamente em seu sistema e, como resultado, você não consegue atender todo o tráfego regular e precisa descartar solicitações de baixa prioridade.

Rate Limit

- O Resilience4j oferece suporte ao rate limiting por meio de um módulo chamado "ratelimiter".
- Existem duas abordagens de rate limit:
 - **Server-side**: visa proteger a aplicação de um DoS(aplicado ao API Gateway ou @Controller).
 - **Client-side**: visa proteger os serviços down-stream da aplicação(aplicado ao @Service)

Rate Limit – Server-side

- O rate limit do lado do servidor (server-side rate limit) no Spring é uma técnica usada para controlar a taxa de solicitações recebidas por um serviço ou API em um servidor.
- O objetivo é limitar o número de solicitações que um cliente pode fazer em um determinado período de tempo.
- O rate limit do lado do servidor pode ser aplicado no Spring Cloud Gateway

Rate Limit – Server-side

- O rate limit do lado servidor no Spring Cloud Gateway usa o algoritmo de **Token Bucket** (balde de tokens):
 - Cada token representa uma permissão para executar uma ação, como uma solicitação em um sistema. O balde é preenchido a uma taxa constante com uma certa quantidade de tokens ao longo do tempo.
 - Quando uma solicitação chega, o sistema verifica se há tokens disponíveis no balde(cada usuário tem um balde). Se houver tokens disponíveis, um token é consumido para permitir o processamento da solicitação. Caso contrário, se não houver tokens disponíveis, a solicitação é rejeitada ou adiada até que haja tokens suficientes disponíveis no balde.
 - A taxa de enchimento do balde determina a taxa na qual os tokens são adicionados ao balde. Por exemplo, se a taxa de enchimento for de 10 tokens por segundo, a cada 0,1 segundo (ou 100 milissegundos), um novo token é adicionado ao balde. No entanto, existe um limite máximo para a capacidade do balde, ou seja, o número máximo de tokens que ele pode conter. Se o balde estiver cheio, os tokens adicionais são descartados.

Rate Limit – Server side

- Para definir um rate limit do lado do servidor deve-se importar no seu projeto do Spring Cloud Gateway o Redis(`spring-boot-starter-data-redis-reactive`) e fazer sua devida configuração.

```
spring:
  redis:
    connect-timeout: 2s
    host: localhost
    port: 6379
    timeout: 1s
```

Time limit for a connection to be established

Default Redis host

Default Redis port

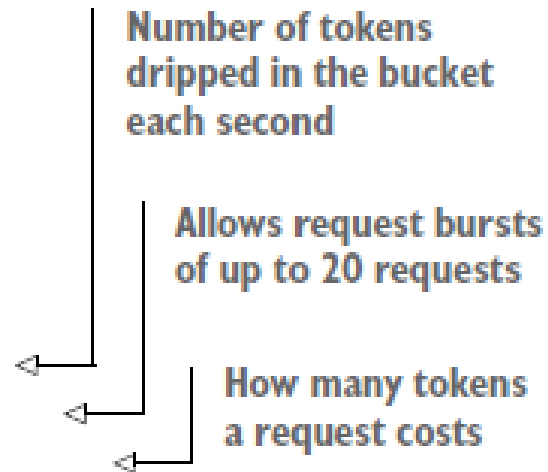
Time limit for a response to be received

Configurando o Rate Limit – Server side

- Vamos configurar o algoritmo para que cada solicitação custe 1 token (**redis-rate-limiter.requestedTokens**).
- Os tokens são adicionados ao balde seguindo a taxa de reposição configurada (**redis-rate-limiter.replenishRate**), que definiremos como 10 tokens por segundo.
- Às vezes, podem ocorrer picos, resultando em um número maior de solicitações do que o usual. Você pode permitir explosões temporárias definindo uma capacidade maior para o balde (**redis-rate-limiter.burstCapacity**), como 20.
- Isso significa que, quando ocorrer um pico, até 20 solicitações serão permitidas por segundo. Como a taxa de reposição é menor do que a capacidade de explosão, explosões subsequentes não são permitidas.
- Se dois picos ocorrerem sequencialmente, apenas o primeiro terá sucesso, enquanto o segundo resultará em algumas solicitações sendo descartadas com uma resposta HTTP 429 - Too Many Requests.

Rate Limit – Server side

```
spring:
  cloud:
    gateway:
      default-filters:
        name: RequestRateLimiter
        args:
          redis-rate-limiter:
            replenishRate: 10
            burstCapacity: 20
            requestedTokens: 1
```



Rate Limit – Server side

- O filtro `RequestRateLimiter` depende de um bean **KeyResolver** para determinar qual balde utilizar para cada solicitação.
- Por padrão, ele utiliza o usuário atualmente autenticado no Spring Security.
- Como não estamos trabalhando com o Spring Security, precisamos definir um bean `KeyResolver` personalizado e fazer com que ele retorne um valor constante (por exemplo, "anônimo"), de modo que todas as solicitações sejam mapeadas para o mesmo balde.

```
@Bean
public KeyResolver keyResolver() {
    return exchange -> Mono.just("anonymous");
}
```

Rate Limit – Server side

- Não há uma regra geral a seguir para definir bons números para o rate limit.
- Você deve começar com os requisitos da sua aplicação e adotar uma abordagem de tentativa e erro: analise o tráfego em produção, ajuste a configuração e repita esse processo até alcançar uma configuração que mantenha seu sistema disponível sem afetar negativamente a experiência do usuário.
- Mesmo após isso, você deve continuar monitorando o status dos seus rate limits, pois as coisas podem mudar no futuro.

Rate Limit – Client- side

- O rate limit do lado do cliente no Resilience4j é uma funcionalidade que permite controlar a taxa de solicitações enviadas por um cliente a um serviço externo.
- Ele ajuda a evitar sobrecarregar o serviço de destino, garantindo que o cliente não exceda um limite pré-definido de solicitações em um determinado período de tempo.

Rate Limit – Client- side

- O Resilience4j oferece duas implementações para o padrão de rate limiter cliente-side:
 - AtomicRateLimiter
 - SemaphoreBasedRateLimiter.
- A implementação padrão para o RateLimiter é o AtomicRateLimiter.

Rate Limit – Client- side

- O SemaphoreBasedRateLimiter é o mais simples.
- Essa implementação é baseada em ter um objeto `java.util.concurrent.Semaphore` que armazena as permissões atuais.
- Nesse cenário, todas as threads de usuário chamam o método `semaphore.tryAcquire` para acionar uma chamada a uma thread interna adicional, executando `semaphore.release` quando um novo `limitRefreshPeriod` é iniciado.

Rate Limit – Client- side

- Por sua vez, o AtomicRateLimiter divide todos os nanossegundos desde o início em ciclos, e a duração de cada ciclo é o período de atualização (em nanossegundos).
- Em cada início de ciclo, devemos definir as permissões ativas para limitar o período.
- Portanto, a configuração do rate limit – cliente-side depende das seguintes configurações:
 - O primeiro parâmetro, **timeoutDuration**, nos permite definir o tempo que uma thread espera pela permissão; o valor padrão para esse parâmetro é de 5 segundos.
 - O segundo parâmetro, **limitRefreshPeriod**, nos permite definir o período que limita o número de invocações. Após cada período, o rate limiter redefine a contagem de permissões de volta para o valor **limitForPeriod**. O valor padrão para o **limitRefreshPeriod** é de 500 nanossegundos.
 - O último parâmetro, **limitForPeriod**, nos permite definir o número de permissões(invocações) disponíveis durante um período de atualização. O valor padrão para o **limitForPeriod** é de 50.

Rate Limit – Client- side

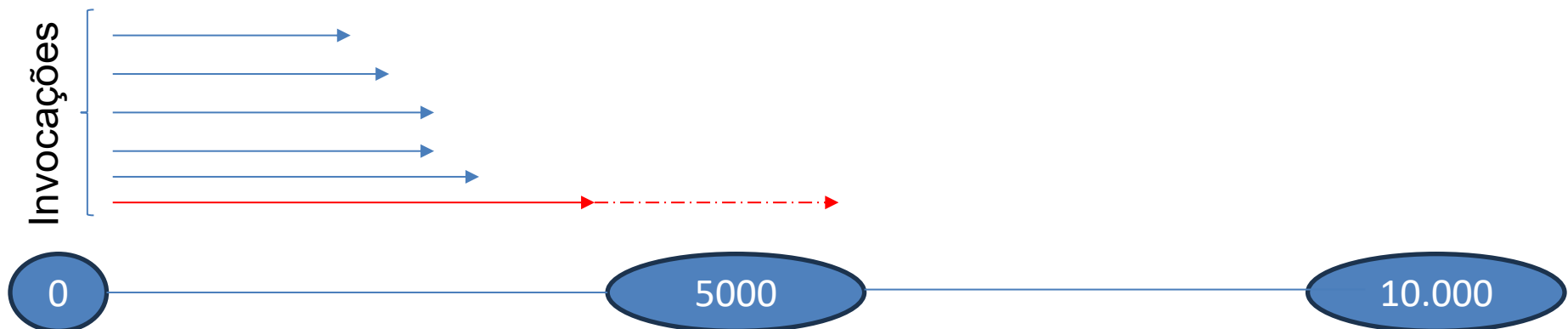
- Neste exemplo, o licenseService poderá realizar 5 invocações num intervalo de 5000 nanoseconds.

```
resilience4j.ratelimiter:  
  instances:  
    licenseService:  
      timeoutDuration: 1000ms  
      limitRefreshPeriod: 5000  
      limitForPeriod: 5
```

Defines the time a thread waits for permission

Defines the period of a limit refresh

Defines the number of permissions available during a limit refresh period



Rate Limit – Client- side

- Para usar o licenseService rate limit é preciso utilizar a anotação no respectivo método.

```
@CircuitBreaker(name= "licenseService",
    fallbackMethod= "buildFallbackLicenseList")
@RateLimiter(name = "licenseService",
    fallbackMethod = "buildFallbackLicenseList")
@Retry(name = "retryLicenseService",
    fallbackMethod = "buildFallbackLicenseList")
@Bulkhead(name= "bulkheadLicenseService",
    fallbackMethod= "buildFallbackLicenseList")
public List<License> getLicensesByOrganization(String organizationId)
    throws TimeoutException {
    logger.debug("getLicensesByOrganization Correlation id: {}",
        UserContextHolder.getContext().getCorrelationId());
    randomlyRunLong();
    return licenseRepository.findByOrganizationId(organizationId);
}
```

Sets the instance name and fallback method for the rate limiter pattern

Combinando Padrões

- Com o Resilience4j, podemos aplicar vários padrões à mesma chamada de método, definindo as anotações para esse método.
- É importante observar que a ordem de execução das funcionalidades do Resilience4j é a seguinte:
 - **Retry (CircuitBreaker (RateLimiter (Bulkhead (Função))))**

Combinando Padrões

```
@CircuitBreaker(name = "servicebeta",
    fallbackMethod= "buildFallbackGetData")
@Retry(name = "retryLicenseService",
    fallbackMethod="retryFallbackGetData")
@Bulkhead(name= "bulkheadservicebeta", type = Bulkhead.Type.SEMAPHORE,
    fallbackMethod= "buildFallbackGetData")
public ResponseEntity getData() {
    return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);
}
public ResponseEntity buildFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action"+t.getMessage());
}
public ResponseEntity retryFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action for retry");
}
```

Mudando a ordem de execução dos padrões

- Se você precisa de uma ordem diferente, é necessário usar o estilo de encadeamento funcional para definir explicitamente a ordem dos aspectos.
- Por exemplo, para fazer com que o Circuit Breaker seja executado após a conclusão do Retry, você deve definir a propriedade "retryAspectOrder" com um valor maior do que o valor de "circuitBreakerAspectOrder" (quanto maior o valor, maior a prioridade).

```
54 resilience4j:  
55     circuitbreaker:  
56         circuitBreakerAspectOrder: 1  
57     retry:  
58         retryAspectOrder: 2  
59
```

Verificando a saúde de um endpoint

- As informações de saúde do Spring Boot Actuator podem ser usadas para verificar o status da sua aplicação em execução.
- Essas informações são frequentemente utilizadas por softwares de monitoramento para alertar alguém caso um sistema em produção tenha problemas graves.

Verificando a saúde de um endpoint

```
13 management.health.circuitbreakers.enabled: true
14 management.endpoint.health:
15   enabled: true
16   show-details: always
17
18 resilience4j.circuitbreaker:
19   configs:
20     default:
21       registerHealthIndicator: true
22   instances:
23     servicebeta:
24       registerHealthIndicator: true
25       ringBufferSizeInClosedState: 5
26       ringBufferSizeInHalfOpenState: 3
27       waitDurationInOpenState: 10s
28       failureRateThreshold: 5
29       recordExceptions:
30         - org.springframework.web.client.HttpServerErrorException
31         - java.io.IOException
32         - java.lang.IllegalStateException
33         - java.util.concurrent.TimeoutException
34         - org.springframework.web.client.ResourceAccessException
--
```

Verificando a saúde de um endpoint

GET

http://localhost:8081/actuator/health

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Query Params

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body

Cookies

Headers (5)

Test Results

Status: 200 OK

Time: 7 ms

Size: 939 B

Sav

Pretty

Raw

Preview

Visualize

JSON

```
1 {
2   "status": "UP",
3   "components": {
4     "circuitBreakers": {
5       "status": "UP",
6       "details": {
7         "servicebeta": {
8           "status": "UP",
9           "details": {
10            "failureRate": "0.0%",
11            "failureRateThreshold": "5.0%",
12            "slowCallRate": "0.0%",
13            "slowCallRateThreshold": "100.0%",
14            "bufferedCalls": 5,
15            "slowCalls": 0,
16            "slowFailedCalls": 0,
17            "failedCalls": 0,
18            "notPermittedCalls": 0,
19            "state": "CLOSED"
20          }
21        }
22      }
23    },
24    "..."
25  }
```

Verificando a saúde de um endpoint

GET

http://localhost:8081/actuator/health

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Query Params

KEY	VALUE	DESCRIPTION	..
Key	Value	Description	

Body

Cookies

Headers (5)

Test Results

🌐

Status: 200 OK

Time: 13 ms

Size: 983 B

Save

Pretty

Raw

Preview

Visualize

JSON

🔧

```
1 {
2   "status": "UP",
3   "components": {
4     "circuitBreakers": {
5       "status": "UNKNOWN",
6       "details": {
7         "servicebeta": {
8           "status": "CIRCUIT_OPEN",
9           "details": {
10             "failureRate": "40.0%",
11             "failureRateThreshold": "5.0%",
12             "slowCallRate": "0.0%",
13             "slowCallRateThreshold": "100.0%",
14             "bufferedCalls": 5,
15             "slowCalls": 0,
16             "slowFailedCalls": 0,
17             "failedCalls": 2,
18             "notPermittedCalls": 1,
19             "state": "OPEN"
20           }
21         }
22       }
23     },
```